

S1

Sémantique SQL – chapitre 1

Anatomie d'une requête

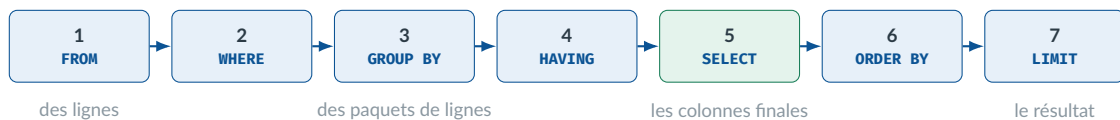
Pourquoi mon alias refuse de fonctionner?? **WHERE** ou **HAVING**???

TU ES ICI SI...

- ▶ « J'ai mis `COUNT(*) > 2` dans mon **WHERE** et PostgreSQL refuse ma requête. »
- ▶ « Je ne sais jamais s'il faut filtrer avec **WHERE** ou avec **HAVING**. »
- ▶ « J'ai donné un nom à ma colonne avec **AS**, mais je ne peux pas m'en servir dans le **WHERE**. »

L'ordre dans lequel on écrit une requête n'est pas celui dans lequel le SGBD l'évalue. Chaque clause reçoit le résultat de la précédente : une clause ne peut donc utiliser que ce qui existe *déjà* au moment où elle s'exécute. Tous les « pourquoi ça ne marche pas » de cette fiche découlent de cette seule idée.

L'ordre réel



Les alias définis dans **SELECT** (étape 5) n'existent donc pas à l'étape 2 (**WHERE**), mais existent bien à l'étape 6 (**ORDER BY**). Les groupes n'existent pas à l'étape 2, mais existent à l'étape 4.

WHERE ou **HAVING**? Les deux, et ce n'est pas redondant

```

SELECT rating, COUNT(*) AS nb
FROM film
WHERE rental_rate > 1    -- j'écarte des FILMS
GROUP BY rating
HAVING COUNT(*) >= 2;    -- j'écarte des GROUPES
  
```

	WHERE	HAVING
filtre des s'exécute	lignes avant GROUP BY	groupes après
agrégats	✗ interdits	✓ obligatoires
alias du SELECT	✗	✗

Sur les 6 films de notre catalogue : le **WHERE** écarte les 2 films à 0,99 €; il reste 4 films, répartis en 3 groupes (R : 2, PG : 1, PG-13 : 1); le **HAVING** ne conserve que R. Résultat : **une seule ligne**, R | 2.

Les trois erreurs classiques

✗ AGRÉGAT DANS LE **WHERE**

```

SELECT category_id, COUNT(*)
FROM film
WHERE COUNT(*) >= 3
GROUP BY category_id;
  
```

✓ CE QU'IL FAUT ÉCRIRE

```

SELECT category_id, COUNT(*)
FROM film
GROUP BY category_id
HAVING COUNT(*) >= 3;
  
```

CE QUE POSTGRESQL VOUS DIT

```

ERROR: aggregate functions are not allowed in WHERE
  
```

✗ ALIAS UTILISÉ TROP TÔT

```
SELECT title, rental_rate * 2 AS prix_double
FROM film
WHERE prix_double > 5;

prix_double n'existe pas encore : le WHERE s'exécute avant le
SELECT.
```

✓ CE QU'IL FAUT ÉCRIRE

```
SELECT title, rental_rate * 2 AS prix_double
FROM film
WHERE rental_rate * 2 > 5      -- on répète l'
                             expression
ORDER BY prix_double;        -- ici l'alias marche
```

CE QUE POSTGRESQL VOUS DIT

ERROR: column "prix_double" does not exist

✗ COLONNE NI GROUPÉE NI AGRÉGÉE

```
SELECT rating, title, COUNT(*)
FROM film
GROUP BY rating;
```

Après regroupement, une ligne représente *plusieurs* films : lequel des titres faudrait-il afficher ?

✓ CE QU'IL FAUT ÉCRIRE

```
SELECT rating, COUNT(*), MIN(title)
FROM film
GROUP BY rating;
```

MIN(title) donne *une* valeur par groupe : la question redevient bien posée.

CE QUE POSTGRESQL VOUS DIT

ERROR: column "film.title" must appear in the GROUP BY clause or be used in an aggregate function

LA RÈGLE DE DÉCISION

Avant d'écrire une condition, demandez-vous **ce qu'elle filtre** :

« Est-ce que je parle d'une ligne de la table, ou d'un paquet de lignes ? »

Une ligne (rental_rate > 1, city = 'Aix') → **WHERE**.

Un paquet (COUNT(*) >= 2, AVG(length) > 100) → **HAVING**.

Et pour un alias : il est né dans le **SELECT**, donc utilisable seulement *après* lui – c'est-à-dire dans **ORDER BY**.

VÉRIFIE QUE TU AS COMPRIS

- Dans quelle clause placez-vous « les films sortis après 2015 » ? Et « les catégories contenant plus de 3 films » ?
- Pourquoi ORDER BY nb DESC fonctionne-t-il alors que WHERE nb > 2 échoue, si nb est un alias défini dans le **SELECT** ?
- Cette requête est-elle correcte ?
SELECT rating FROM film GROUP BY rating HAVING rental_rate > 1;
- Un camarade écrit HAVING COUNT(*) >= 2 sans **GROUP BY**. Est-ce une erreur ?

RÉPONSES

1. **WHERE** pour les films (condition sur une ligne), **HAVING** pour les catégories (condition sur un groupe). **2.** Parce que **ORDER BY** s'exécute *après* le **SELECT**, où l'alias est créé, alors que **WHERE** s'exécute *avant*. **3.** Non : rental_rate n'est ni dans le **GROUP BY** ni agrégé, le **HAVING** ne sait pas de quel film il parle. Il fallait soit filtrer en **WHERE**, soit écrire HAVING MIN(rental_rate) > 1 – ce qui ne veut pas dire la même chose. **4.** Non, c'est accepté : sans **GROUP BY**, toute la table forme un seul groupe. La requête renvoie donc 0 ou 1 ligne selon que la condition est vraie.